State-Level Provenance over Upsert Sources

> **Implemented and wired into the demo** as root README §3.5.3 — `make cp-flink-reports-up ENABLE_STATE_PROVENANCE=true` on CP, `make cc-flink-reports-up ENABLE_STATE_PROVENANCE=true` on CCAF. Off by default on both, and both derive **identical version IDs** for the same record (§5.0). Code: `StateProvenancePTF`, `StateVersion`, `09_state_provenance_sinks.fql`, `85_state_provenance.fql`, `setup-ccaf-state-provenance.tf`.

Table of Contents

- 1.0 What "state-level" actually changes
- 2.0 The model: content-addressed versions
  - 2.1 Version identity
  - 2.2 Parents travel with the version, not beside it
  - 2.3 Edges are a projection, not a source of truth
- 3.0 Why the four failure modes disappear
- 4.0 Implementation
  - 4.1 Read the log, own the state
  - 4.2 The PTF
  - 4.3 The SQL
  - 4.4 Sinks
  - 4.5 Running it
- 5.0 CP vs CCAF: where the divergence actually lands
- 6.0 Limits, and what to verify first

1.0 What "state-level" actually changes

flink-collector.md §3.1 draws the line this document starts from: an isotope records an **itinerary** — one identity, ordered hops, a *path* — and that is truthful only while every step is 1:1. State-level lineage is never 1:1. A row's current value is derived from every change that ever touched it, which is a **DAG over versions**, not a path over messages.

So the first design decision is to stop overloading the isotope. In state mode, `ISOTOPE_APPEND_HOP` **is not used at all.** Asking "is a −U/+U pair one hop or two?" has no good answer because the question is wrong: a revision is not a movement. The hop list stays what it is — a message-level artifact riding the log — and state provenance becomes a second, parallel record keyed by *version* rather than by trace. The two coexist without either lying.

The four problems that make the windowed merge collector unworkable on upsert sources — no window to key on, two statements drifting apart, changelog-blind projections, and time-zone-dependent bounds — are all symptoms of running a path model over a changelog. Change the model and they stop being problems rather than getting patched.

2.0 The model: content-addressed versions

2.1 Version identity

Every emitted state of a row gets a `version_id` derived from its own content:

```
digest      = SHA-256( source_name || SEP || entity_key || SEP || content )
version_id  = event_time_ms (48 bits) || digest[0:10]   →  patched to a
valid UUIDv7
```

`SEP` is the same NUL field separator `MergeTrace` already uses, with the same null marker discipline. The high 48 bits carry event time exactly as `Isotope.uuidV7Bytes` and `MergeTrace` lay them out, so version IDs sort chronologically and interleave with interceptor-minted trace IDs rather than being 16 opaque hashed bytes.

`content` is the raw value **without its Schema Registry header**. A registry serializer starts every value with a magic byte `0x00` and a 4-byte schema ID, and that ID is assigned by whichever registry the producer used. CP's registry and Confluent Cloud's give the same schema different IDs, so hashing those four bytes would give the same record a different version on each runtime. `StateVersion.canonicalContent` drops the first five bytes whenever a value starts with `0x00` and is at least five bytes long, and hashes the rest. Protobuf's message-index byte comes after the schema ID and stays in the hash, because it comes from the schema, not from the registry. A value written with the schema ID in a Kafka header has no prefix to strip, so it hashes to the same version as the prefixed form. The rule checks only the byte layout, because nothing in a `BYTES` column says which serializer wrote it. So for a non-registry payload that happens to start with `0x00`, two values that differ only in bytes 1–4 would share a version. Every source in this project is registry-framed.

That makes identity **(event time, content)**, not content alone — worth stating plainly. Two byte-identical states at the same event time are the same version, which is the idempotence that actually matters under at-least-once delivery; a row that revisits an earlier value *later* is a distinct version, which is what a version chain should say. The preimage is a wire contract: `StateVersionTest` pins field order, separator forgery, null-vs-empty, tombstones, the Schema Registry strip, and the UUIDv7 layout, so changing any of them is a build failure rather than silently renumbered history.

Content addressing buys four properties that matter more here than they did for the windowed merge:

- **No window required.** Identity comes from the value, so unbounded `GROUP BY`, regular joins, and dedup all have something to key on — the thing the window was standing in for.
- **Deterministic across runtimes and across reprocessing.** CP and CCAF derive the same ID from the same content, because the one part of the value that differs between their registries is stripped first; a replay six months later derives it again.
- **Idempotent under duplicate emission.** A retract stream revises the same logical row repeatedly, and at-least-once delivery repeats records. Recomputing a version you already published yields the same ID, so the provenance topic tolerates duplicates by construction.
- **Retraction becomes append.** You never mutate a version; you publish a new one. The provenance stream is therefore **append-only even though its subject is an updating table** — which is what lets every operator in the pipeline stay append-only.

A row that reverts to a previous value produces a previously seen ID. That is a deliberate property: identical content is the same state. If they need to distinguish "reverted to X" from "was X," carry `prev_version_id` on the record — the chain disambiguates, the identity doesn't have to.

2.2 Parents travel with the version, not beside it

Each provenance record carries its parent set **inline**:

Column	Carries
`version_id`	This state's content-addressed identity
`entity_key`	The upsert PK this version belongs to
`source_name`	The table this state was observed on — part of the preimage, so the same payload on two tables is two versions
`op`	`UPSERT` / `DELETE`, from the change that produced it
`parents`	`ARRAY<STRING>` of the versions this one was derived from. In a 1:1 stage that is the version it supersedes, so the chain is `parents[0]` — no separate `prev_version_id` column, because a redundant one is a column that can disagree
`parent_overflow`	Parents dropped past the inline cap, `0` normally
`emitted_at`	Event time of the change

One record, one operator, one state. The merged output and its parent set can no longer disagree, because they are not two things.

2.3 Edges are a projection, not a source of truth

If a flat edge table is wanted for joins, derive it *downstream* of the provenance topic:

```
INSERT INTO state_provenance_edges
SELECT `version_id`, `entity_key`, `parent`, `emitted_at`
FROM state_provenance
CROSS JOIN UNNEST(`parents`) AS t(`parent`);
```

This is a pure projection of an already-consistent record, over an append-only topic. It cannot drift from what it flattens — unlike `81_merge_edge_markers.fql`, which is a second statement racing the first over the same source.

3.0 Why the four failure modes disappear

Objection to the windowed design	Why it does not apply here
Unbounded `GROUP BY` / regular join / dedup have nothing to key on	Identity is derived from content, not from window bounds. No window is needed for any of them.

Objection to the windowed design	Why it does not apply here
Two statements, two watermarks, `event_count 1000` vs `edge_list 1001`	There is one statement and one operator. The parent set is a column of the record it describes; drift is not expressible.
The `isotope` view is changelog-blind	Nothing consumes a changelog. The pipeline reads the physical log append-only and reconstructs state inside a PTF (§4.1), so no operator ever sees an update.
`UNIX_TIMESTAMP(CAST(window_start AS STRING))` is time-zone-dependent	No window bounds appear in any identity. Event time is carried as epoch millis end to end, never round-tripped through a formatted string.

4.0 Implementation

4.1 Read the log, own the state

The move that makes everything else work: **do not consume the upsert view — consume the log underneath it, in append mode, and rebuild state yourself inside the PTF.**

Flink's restrictions on updating streams (window TVFs refuse them, time attributes do not survive them, plain sinks reject them) all vanish, because nothing in the pipeline is an updating stream. A compacted topic's messages are already an append-only sequence of changes; a Debezium topic is even more explicitly so. The isotope's headers are present on every one of those physical messages regardless of how any other consumer chooses to interpret the topic — including on tombstones, which means **deletes get first-class provenance**, something a state view cannot give you.

The cost is that you are now responsible for the state machine that `upsert-kafka` would have run for you. That is the trade: you take on state management and get changelog-free planning, deterministic identity, and consistent multi-output emission in exchange.

4.2 The PTF

`StateProvenancePTF` follows the idiom `StuckTracePTF` established — a `@StateHint` POJO, a `SET_SEMANTIC_TABLE` + `REQUIRE_ON_TIME` input, `ctx.timeContext(Instant.class)` for event time. Per entity it holds the current version ID, the parents folded in since the last publication, and the versions not yet published. Per input row it derives the new version, drops it if it is the current version or already waiting (a redelivery is not a new state), parks it, and registers an event-time timer at its event time. The timer publishes every parked version at or before the watermark, **oldest first**, each naming the one before it as its parent.

Why the timer. `REQUIRE_ON_TIME` gives the function each row's event time; it does not sort the rows. `entity_log` is a `UNION ALL` of one table per topic, so an entity's records reach the PTF in whatever order the source readers deliver them. The first version of this function published on arrival, and on the cluster its chains came out as `enriched ← fulfilled ← placed` and every other permutation. Waiting for the watermark means nothing earlier can still be in flight. The version ID leads with 48 bits of event time, so sorting IDs sorts by event time, and ties within one millisecond break on the digest the same way every run. The costs:

- **Latency:** a version is published only after the watermark passes it, which is the source's 5s bounded out-of-orderness.
- **Late records:** a record that arrives behind the watermark is still published, on the next watermark, but after versions already on the chain.
- **Tests:** `StateProvenancePTFTest` pins the ordering, redelivery and per-entity independence on a local MiniCluster.

Three constraints worth knowing before writing it, all learned the hard way on this project:

- **CCAF rejects `MapView`/`ListView` in PTF state.** A plain `Map`/`List` field is the documented replacement — and it *still* rejects a `byte[]` map **value**, where a Base64 `String` works. Both fail at `CREATE FUNCTION` time, not at runtime, so a local CP test will not catch them.
- **Emit one row type.** cp-flink 2.1.2's Expander round-trip rejects `CREATE VIEW` over a PTF, which is why the existing PTFs are invoked from `INSERT INTO` directly. Emitting state rows and edge rows as one tagged stream to be split by a view is therefore not available — another reason the parent set is a column rather than a second output.
- **Bound the parent set.** `PARENT_CAP` plus `parent_overflow` keeps the record and the state bounded. Entity-scoped derivations sit far below any sane cap; a global aggregate will blow through it, and for that case carry a count and a sketch rather than the members.

4.3 The SQL

One statement per derived table, on both runtimes:

```
INSERT INTO `isotope_state_provenance`
SELECT `version_id`, `entity_key`, `source_name`, `op`, `parents`,
`parent_overflow`, `emitted_at`
FROM TABLE(
    STATE_PROVENANCE(
        input   => TABLE `entity_log` PARTITION BY `entity_key`,
        on_time => DESCRIPTOR(`event_time`),
        uid     => 'state-provenance-v2'
    )
);
```

`entity_log` (`09_state_provenance_sinks.fql`) is the append-mode change log over the order topics, which are already declared `value.format = 'raw'` with headers as `VIRTUAL` columns — indifferent to value format, and tolerant of tombstones. On CCAF the same view lives in `setup-ccaf-state-provenance.tf`, with two substitutions that come from the Topic Catalog rather than from the design: the raw value column is `val` rather than `value`, and event time is `$rowtime` rather than a declared `event_time`.

The demo's entity key. A real deployment keys on the upsert PK. This demo has none to offer: `App.java` sets the record key to the producing *service* name and `hop` forwards the DemoEvent value verbatim, so neither identifies an entity. The isotope trace ID does — stamped once at the origin, surviving every hop — so the demo treats one traced order as the entity and its three stage records as three versions of it. Because `source_name` is in the preimage, those are three distinct versions even though the payload bytes never change:

```
orders.placed → orders.enriched → orders.fulfilled
```

each naming its predecessor in `parents`. That is the same shape a real upsert entity produces.

4.4 Sinks

- **`isotope_state_provenance`** — append-only. Plain `kafka` sink on CP (`avro-confluent`), append mode with `proto-registry` on CCAF. Retention is unlimited on both: the CP deploy script sets `retention.ms=-1`, and the CCAF `CREATE TABLE` sets `'kafka.retention.time' = '0'` (CCAF's way of saying unlimited, since Flink durations don't accept `-1`). This is the durable lineage record.
- **`state_provenance_edges`** — append-only, the §2.3 `UNNEST` projection. Not created by the demo; derive it if you want flat edges.
- **`current_state`** — the only upsert table in the design, and it is a *convenience view* for "what is order 42 now," keyed by `entity_key`. Nothing in the provenance path reads it, so its changelog mode infects nothing.

4.5 Running it

```
make cp-flink-reports-up ENABLE_STATE_PROVENANCE=true

make cc-flink-reports-up ENABLE_STATE_PROVENANCE=true \
    CONFLUENT_API_KEY=$CONFLUENT_API_KEY
CONFLUENT_API_SECRET=$CONFLUENT_API_SECRET

# both collectors — they answer different questions, and neither implies
the other
make cp-flink-reports-up ENABLE_MERGE_PROVENANCE=true
ENABLE_STATE_PROVENANCE=true
```

Off, nothing is created: no view, no topic, no statement, and the deployment is byte-identical to a build without the feature. Teardown removes `isotope_state_provenance` whether or not the flag was passed, so a `down` run cannot strand a topic an earlier `up` created.

5.0 CP vs CCAF: where the divergence actually lands

The feature runs on both runtimes and gives the same version IDs on both. Getting there took two facts and one rule.

- **Both runtimes can read the raw value.** CP's source tables are declared `'value.format' = 'raw'`, so the value is a `BYTES` column called `value`. On CCAF, Terraform creates the order topics before any producer registers a schema, so the Topic Catalog imports them as `(key BYTES, val BYTES)` (flink-collector.md §2.2 records the `DESCRIBE`). That is the same bytes under a different column name, so the PTF hashes the same input on both.
- **The bytes still differ in one place: the registry's schema ID.** The services write `KafkaProtobufSerializer` values, which start with a magic byte and a 4-byte schema ID. Each

runtime has its own registry, so the same `DemoEvent` carries a different ID on CP and on Confluent Cloud.
- **So `StateVersion` strips that header before hashing** (§2.1). What's left is the Protobuf message index and the payload, and both come from the schema and the record alone. `StateVersionTest` checks this directly: the same payload under two different schema IDs is the same version.

An earlier draft of this section assumed CCAF exposed only typed Protobuf columns, and proposed hashing a shared rendering of those columns. That would have worked, but it adds a second definition of "content" to keep stable. It also isn't needed while the order topics are imported as bytes.

One condition to keep: the order topics must stay byte-imported on CCAF. If one of them is created *after* its value schema is registered, the Topic Catalog derives typed columns, there's no `val` column, and the `entity_log` view stops planning. It fails loudly rather than computing different IDs. `setup-cc-kafka.tf` already pre-creates those topics for the always-on collector, which reads `val` too.

Nothing else differs at the capability level. `StateProvenancePTF`'s state uses plain `String`/`List`/`Map` fields so that CCAF accepts it, and on CCAF it's registered from the same shadow JAR with `CREATE FUNCTION … USING JAR`. The design also avoids the one place the runtimes really do differ: CCAF has `changelog.mode = retract`, and stock open-source Flink has no Kafka sink that supports retractions. Because versions are content-addressed and never changed, what would have been a retract stream is an append-only stream, so retract semantics are never needed.

On CCAF, `STATE_PROVENANCE` is registered only when the feature is on, unlike the merge UDFs. CCAF checks a PTF's state type at `CREATE FUNCTION` time. If that check ever rejected this class, an always-on registration would break the default deploy, including the seven reports, even with the feature off.

The remaining differences are in how tables are declared, the same split this project already documents for headers in flink-collector.md §2.2:

	CP / open-source Flink	**CCAF**
Source table in append mode	own the DDL: `'connector' = 'kafka'`, raw `value`	Topic Catalog auto-imports `(key, val)` BYTES in append mode; no `ALTER` needed
Provenance sink	`kafka` + `avro-confluent`, `retention.ms=-1` set by the deploy script	append mode + `proto-registry`, `'kafka.retention.time' = '0'` in the DDL
`current_state` table	`upsert-kafka` + PK	`changelog.mode = upsert` + `PRIMARY KEY … NOT ENFORCED`
The PTF	same shadow JAR, registered programmatically	same shadow JAR, `CREATE FUNCTION … USING JAR`
PTF state	`MapView`/`ListView` allowed	plain `Map`/`List` only; no `byte[]` map values

The last row is a coding constraint, not a behavioral difference: write the state POJO to CCAF's rules and the identical class runs on both. That is the same discipline the existing two PTFs already follow.

6.0 Limits, and what to verify first

Verify before building:

1. **Flipping `changelog.mode` on a shared CCAF catalog table** affects every consumer of that table. The demo doesn't need to, because its order topics are imported in append mode. A real upsert source on CCAF would be imported in upsert mode, and then either flipping it or a second append-mode table over the same topic is needed. Confirm that CCAF allows that in your environment.
2. **Writable `headers` metadata on an upsert-mode sink**, if they ever want the collector writing to one. Not needed for this design; needed if message-level tracing is layered on top.
3. **Canonicalization stability** across the language boundary if anything outside Flink computes a `version_id`. Two implementations of "canonical content" is two chances to disagree, and that includes the Schema Registry strip in §2.1: any other implementation must drop the same five bytes under the same condition.

Accepted limits:

- **Ancestry is queryable, not traversable — in Flink.** Flink SQL has no recursive CTEs, so walking more than one generation belongs in a relational or graph store fed by the provenance topic — the same boundary flink-collector.md §3.1 already sets for merge edges. The opt-in Iceberg lake is that store: `ENABLE_LAKE=true` copies this topic into `lake.isotope.state_provenance`, and `scripts/lake-query.sh` walks every chain with `WITH RECURSIVE`.
- **Compaction bounds re-derivation, not lineage.** Compacting the source topic removes superseded messages, but every version already emitted keeps its place in the graph. It lives in `isotope_state_provenance`, with its parents inline. What compaction does remove is the *content* of old versions, since the provenance record holds IDs and no payload. That costs three things: seeing what an old version's value was, recomputing a `version_id` from its bytes to verify it, and rebuilding the provenance from `earliest-offset` after the PTF logic changes. Only the last is operational, and only if you ever need to rebuild the graph rather than read it. If you do, archive the raw source log somewhere that is not compacted, such as an Iceberg table.
- **The provenance topic's retention is the real bound on lineage.** On a broker with default settings, a topic created without explicit configs uses `cleanup.policy=delete` and about 7 days of retention, so the whole graph would expire. The CP deploy script pins `isotope_state_provenance` to `retention.ms=-1`, `retention.bytes=-1` and `cleanup.policy=delete`, which means infinite retention and no compaction. CCAF's `CREATE TABLE` sets the same thing with `'kafka.retention.time' = '0'` and `'kafka.cleanup-policy' = 'delete'`. Compaction would be wrong here as well: it keeps one record per key, and a version chain needs all of them. For history that you can query, and traverse with recursive CTEs, land the topic in Iceberg.
- **Global aggregates do not fit.** The inline parent set is right for entity-scoped derivation and wrong for a `SUM` over everything. That case wants a count plus a sketch, and loses the ability to name the parents.
- **Idle partitions stall event time.** Upsert topics are frequently bursty and partially idle; the 5s watermark delay tuned for the demo is not a starting point for their volumes.